



Input

Specification

Datra and file format

**Document version 2022
October 2022**

Service & Technical Support
samg@scai.fraunhofer.de

**Fraunhofer Institute SCAI
Schloss Birlinghoven 1
53757 Sankt Augustin
Germany**

 **Fraunhofer**
SCAI

Purpose of this document

This short document specifies the **data structure** used to describe a (sparse) linear system of equations, $Au=f$, within the context of SAMG. It also specifies a **file interface** that is the preferred choice for an external user to pass exemplary linear systems of equations to the SAMG Team in case technical support is requested (such as an evaluation or optimization of SAMG's performance).

If requested, ways for transferring data can be provided by the SAMG team. For any technical support, questions, suggestions and the like, do not hesitate to contact us via our service e-mail,

samg@scai.fraunhofer.de¹.

Your SAMG-Team

Remarks on highlighting

  	These symbols are used to draw the reader's attention to particularly important aspects/remarks of this document (order given by increasing importance).
	This symbol marks features which are either not yet installed in the current release or which are in a test phase and not yet supposed to be selected by the average user.

Colored text is used for highlighting as follows:

- Generally, the color dark blue highlights headlines and important aspects.
- Examples, computer output, program code, program and file names are printed in blue.
- Text printed in green refers to options or features which are under development and not yet meant to be used by an average user

¹This e-mail addresses the whole SAMG Team. Therefore, it should not be used to transmit MBytes of data!

1	Notation and Conventions	4
2	Matrix Data Structure	5
2.1	General Sparse Linear System of Equations	5
2.2	Additional Data and Requirements in Case of Coupled Systems	6
2.2.1	Relation Between Variables and Unknowns	6
2.2.2	Relation Between Variables and Points (optional).....	6
2.3	Complex-Valued Linear Systems	6
2.4	Matrix Input Data: Overview	7
3	Additional Information in the MPI Parallel Case	8
3.1	Global Numbering: Minimal Additional Input via SAMGp_pcrs	8
3.2	Local Numbering: Full Control on Matrix Overlap via SAMGp	9
3.2.1	Example of Input Data in the Local Numbering Case with SAMGp	10
4	File Interface: Main Files.....	13
4.1	Format of Writing: Formatted or Binary	13
4.2	Scalar Systems.....	13
4.2.1	Mandatory Files.....	13
4.2.2	Optional Files.....	14
4.2.3	Example	14
4.3	Coupled Systems	15
4.4	Distributed Data in MPI Parallel Runs.....	15
4.5	Coordinates	16
5	Index	17

1 Notation and Conventions

For solving linear systems of equations, $Au=f$, SAMG provides various solution approaches. This is true, in particular, if the linear system corresponds to a discretized *coupled* system of PDEs. We need to clearly distinguish such applications from those derived from *scalar* PDEs. In order to avoid confusion, we need to introduce some basic notation.

We distinguish between what we call **variables**, **unknowns** and **points**:

- **Variable:** Any of the solution components u_i of the linear system $Au=f$ to be solved. Variables are numbered $1,2,\dots,nnu$.
- **Unknown:** Any of the (scalar) physical functions being approximated (eg, pressure, temperature, a velocity component, or a component of the displacement). Assuming $nsys$ such functions to be involved in a given linear system of equations, the unknowns are numbered $1,2,\dots,nsys$.
- **Point:** Location in space where a single variable or a group of variables (corresponding to different unknowns) is defined. Points are generally numbered $1,2,\dots,npnts$.

Remark: Since we usually have the solution of PDEs in mind, we here think of points as being real physical points in space. However, we want to point out that, from SAMG's point of view, it is not important whether "points" really correspond to physical points. Generally, instead of physical points, one may think of the **nodes of a graph** representing the connectivity structure of A .



In general, each variable is associated with both a point and an unknown.

We distinguish between **scalar systems** and **coupled systems**:

- **Scalar system:** We call a linear system of equations, $Au=f$, a "scalar system" if it corresponds to a discretized *scalar* PDE. In particular, there is *only one* unknown involved, that is, all variables correspond to the same physical function (eg, the pressure).
- **Coupled system:** We call a linear system of equations, $Au=f$, a "coupled system" if it corresponds to a discretized *coupled system* of PDEs. In particular, there is *more than one* unknown involved, that is, $nsys>1$.

2 Matrix Data Structure

In this section, we describe the data structure used by SAMG in solving (sparse) linear systems of equations, $Au=f$. We also describe which additional information has to be provided in case of *coupled* systems.

2.1 General Sparse Linear System of Equations

The entries of the matrix A , denoted by $A(i,j)$, are assumed to be stored in **compressed row format** in a vector a , row by row, *each row starting with its diagonal element*. While *all* diagonal entries have to be stored independent of whether or not they are zero, the off-diagonal entries need to be stored only if they are non-zero². Since the order in which the off-diagonal entries are stored in a is arbitrary, two integer pointer vectors ia and ja are required to identify each element in a . Details are described in the following.

If nnu denotes the number of rows (variables), the non-zero entries of the i -th row ($1 \leq i \leq nnu$) are stored in $a(j)$ where

$$ia(i) \leq j \leq ia(i+1)-1.$$

In particular, according to the above-mentioned convention about the location of the diagonal element, $a(ia(i))$ contains the diagonal entry of row i . Note that $ia(1) = 1$ and $ia(nnu+1) = nna+1$ where nna denotes the total number of matrix entries stored.

The pointer vector ja has to be defined so that $ja(j)$ ($1 \leq j \leq nna$) equals the original matrix' column index of $a(j)$, ie, $a(j)$ corresponds to the variable $u(ja(j))$. In particular, since $a(ia(i))$ contains the diagonal entry of row i , we have $ja(ia(i))=i$.

Summarizing, for any $1 \leq i \leq nnu$ and $ia(i) \leq j \leq ia(i+1)-1$, we have $a(j)=A(i,ja(j))$ and the i -th equation of $Au=f$ reads:

$$\sum_{j1 \leq j \leq j2} a(j) u(ja(j)) = f(i)$$

where $u(i)$ and $f(i)$ denote the i -th component of u and f , respectively, and $j1=ia(i)$, $j2=ia(i+1)-1$.

Generally, we assume **typical diagonal entries to be positive**³. This is a pre-requisite for classical AMG components such as variable-wise relaxation and standard interpolation schemes. For the more advanced modules, positive diagonals are formally not necessarily required any more. However, whether or not such problems can be solved efficiently by AMG depends on the application and it is difficult to give some general rules here.

Requirements: Note that ignoring one or more of the following requirements is the reason for most user errors in applying the above data structure!

- Note that $ia(i)$ has to be defined for all $1 \leq i \leq nnu+1$!
- The order of rows has to be such that the i -th row (equation) corresponds to the i -th variable.
- The order of entries within each row is arbitrary except that the diagonal entry has to be first.
- For symmetric matrices A , *all* (non-zero) entries need to be stored (not just a triangular part).



² Note that not all off-diagonal matrix entries stored in a necessarily have to be non-zero. In fact, it makes sense to leave some zero entries in a , for instance, entries which are not zero for reasons of discretization (according to the sparsity pattern) but rather happen to be zero only accidentally.

³ Of course, this assumes a proper scaling of the given matrices. In particular, definite problems should be scaled so that $(Au,u)>0$ for $u \neq 0$.

2.2 Additional Data and Requirements in Case of Coupled Systems

This section is only relevant, if $Au=f$ is a **coupled system** corresponding to unknowns $1,2,\dots,nsys$ with some $nsys>1$. In this case, SAMG needs up to two additional vectors in order to be able to properly distinguish between different unknowns.

2.2.1 Relation Between Variables and Unknowns

For all coupled systems, SAMG needs to know the correspondence between variables and unknowns. This information needs to be provided via an integer **variable-to-unknown pointer vector**, iu : For each variable $1\leq i\leq nnu$, $iu(i)$ has to contain the number of the unknown this variable represents:

$$iu: [1,\dots,nnu] \rightarrow [1,\dots,nsys] .$$

Each unknown has to be represented by (at least) one variable. That is, for each unknown $1\leq k\leq nsys$, we require that $iu(i)=k$ for (at least) one i . Besides what has been described in the previous section, there are no additional requirements on the ordering of variables (rows).

2.2.2 Relation Between Variables and Points (optional)

For an efficient SAMG solution, it is often enough to just provide the vector iu as described above. In general, however, more efficient SAMG strategies might be available if the existence of a point structure can be exploited in addition. To this end, two additional requirements have to be satisfied: First, the variables (rows) have to be **ordered pointwise**. That is, the first variables are those sitting at point 1, followed by those sitting at point 2, etc. Second, in addition to the previous variable-to-unknown pointer iu , a **variable-to-point pointer vector** ip is required, providing information at which point a variable is located. More precisely, assuming the points to be numbered $1,2,\dots,npnts$, for each variable $1\leq i\leq nnu$, $ip(i)$ has to contain the number of the point where this variable is located:

$$ip: [1,\dots,nnu] \rightarrow [1,\dots,npnts] .$$

Naturally, none of the unknowns is allowed to be represented at a point more than once. On the other hand, not all unknowns need to be represented at a point. In general, the number of variables may vary from point to point and there may even be points with no variables attached at all.

2.3 Complex-Valued Linear Systems

SAMG operates on **real-valued** problems only. To use SAMG for **complex-valued** problems, one possibility is to re-write a complex-valued problem, $Au=f$, as a coupled system for the real and imaginary parts. By denoting $A = A_1 + iA_2$, $f = f_1 + if_2$ and $u = u_1 + iu_2$, the following coupled systems are obtained:

$$\begin{pmatrix} A_1 & -A_2 \\ A_2 & A_1 \end{pmatrix} \begin{pmatrix} u_1 \\ u_2 \end{pmatrix} = \begin{pmatrix} f_1 \\ f_2 \end{pmatrix} \quad \text{or} \quad \begin{pmatrix} A_2 & -A_1 \\ A_1 & A_2 \end{pmatrix} \begin{pmatrix} u_2 \\ u_1 \end{pmatrix} = \begin{pmatrix} -f_1 \\ f_2 \end{pmatrix} .$$

Refer to the previous section on how to represent such systems in SAMG's data structure.

2.4 Matrix Input Data: Overview

The following table provides an overview on all input data that are used to pass the linear system of equations to SAMG (matrix, solution vector and right hand side).

Parameter	
Problem size and type	
nnu	Integer. Number of variables, ≥ 1 .
nna	Integer. Number of matrix entries stored in vector a , $\geq nnu$.
The linear system of equations to be solved: $Au=f$	
ia, ja, a	Integer vectors $ia(1:nnu+1)$, $ja(1:nna)$; double vector $a(1:nna)$. Compressed row storage representation of matrix A ($\rightarrow$ Section 2.1). Remark: Upon return, the ordering of ja and a may have changed. The matrix is logically the same, though.
f	Double vector $f(1:nnu)$, right hand side.
u	Double vector $u(1:nnu)$, <i>Input (optional):</i> First guess, if a user-specified first guess shall be used. <i>Output:</i> Upon return, u will be overwritten by the solution of $Au=f$.
Problem type (scalar or coupled system)	
nsys	Integer. Number of unknowns, $1 \leq nsys \leq 99$. $nsys=1$ and $nsys>1$ distinguish scalar and coupled systems, respectively.
Variable-to-unknown and variable-to-point pointers	
iu, ndiu	Integer vector $iu(1:ndiu)$, variable-to-unknown pointer ($\rightarrow$ Section 2.2.1 ²). Only relevant if $nsys>1$ in which case $ndiu=nnu$. Otherwise, vector iu is a dummy vector of length $ndiu=1$.
ip, ndip	Integer vector $ip(1:ndip)$, variable-to-point pointer ($\rightarrow$ Section 2.2.2). Only relevant if $nsys>1$ and if any of the point-based approaches is to be used, in which case $ndip=nnu$. Otherwise, vector ip is a dummy vector of length $ndip=1$.

3 Additional Information in the MPI Parallel Case

If you are using SAMGp in an MPI parallel environment, each process only deals with a sub-part of the full linear system. Only these sub-parts are passed to SAMG by each process. Nevertheless, SAMG solves the global linear system. Therefore, certain additional information are required to allow SAMG to handle matrix overlaps between these different process partitions. Depending on whether your matrix data are available in global or in local numbering, you may prefer either the `samgp_pcrs` or the `samgp` interface, respectively.

3.1 Global Numbering: Minimal Additional Input via SAMGp_pcrs

If the matrix sub-parts per process are available in a global numbering, the efforts in providing information about the matrix overlap between process partitions can be minimized. SAMG then only needs to know how of how many matrix rows this overlap area does consist. It can even be advised to self-compute this value, if it was not known.

The matrix data themselves are passed via the same data structures as in the non-MPI case (→ Section 2.4). The array sizes and the expected data format does slightly change, though. An overview is provided in the table below.

Parameter	Explanation
Local problem size	
nsys	Integer. Number of unknowns in the given system over all processes , ≥ 1 .
nnu	Integer. Number of (local) variables, ≥ 1 .
nna	Integer. Number of matrix entries stored in the (local) vector a , $\geq nnu$.
Partition Overlap Information	
nrhalo	Integer. Specifies the size of the matrix overlap (number of matrix rows that belong to other process partitions and where there are non-zero couplings to from rows of the local partition). If this size is not known, use -1 and SAMG will self-compute the value. The dimension of u , f , iu and ip is then adjusted internally to this value. The user will not need to care.
Local part of the linear system to be solved	
ia, ja, a	Integer vectors $ia(1:nnu+1)$, $ja(1:nna)$; double vector $a(1:nna)$. CSR representation of local part of matrix A. Note: $nna=ia(nnu+1)-1$. Input: $ja(j)$ has to contain global indices! Output: $ja(j)$ will contain local indices!
u	Double vector $u(1:nnu+\max(0,nrhalo))$, local solution vector. <i>The auxiliary space ($nnu+1$ to $nnu+nrhalo$) does not need to be initialized.</i>
f	Double vector $f(1:nnu+\max(0,nrhalo))$, local part of the right hand side. <i>The auxiliary space ($nnu+1$ to $nnu+nrhalo$) does not need to be initialized.</i>
Local variable-to-unknown and variable-to-point pointers	
iu, ndiu	Integer vector $iu(1:ndiu)$, variable-to-unknown pointer. Only relevant if $nsys>1$ in which case $ndiu=nnu+\max(0,nrhalo)$. Otherwise, iu is just a dummy vector and ndiu=1 .
ip, ndip	Integer vector $ip(1:ndip)$, variable-to-point pointer. Only relevant if $nsys>1$ and if any of the point-based approaches is to be used. Then $ndip=nnu+\max(0,nrhalo)$. Otherwise, ip is just a dummy vector and ndip=1 .

3.2 Local Numbering: Full Control on Matrix Overlap via SAMGp

If the matrix sub-parts are available in local numbering per process, the user has to take care on the matrix overlap handling himself. These overlap information are typically available anyway then. They just need to be passed to SAMGp. For this purpose, several additional input data structures are used. The local matrices themselves are still passed via the same data structures as in the non-MPI case (→ Section 2.4). Just the array sizes may slightly differ.

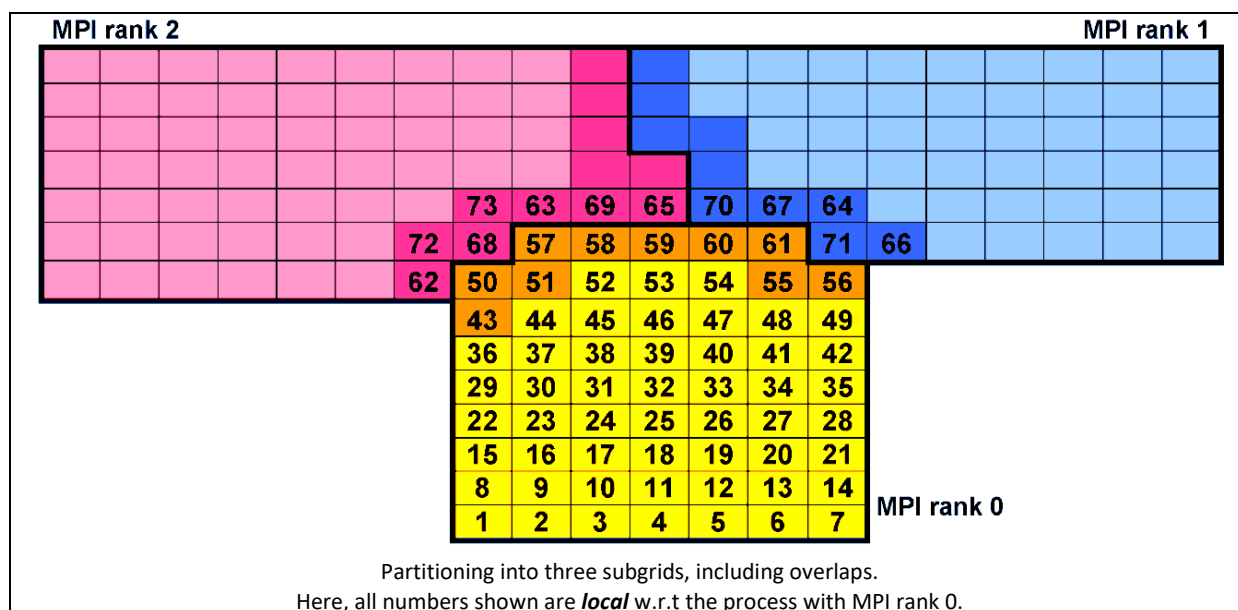
An overview on the full set of input data structures is provided in the table below. An illustrative example is provided afterwards.

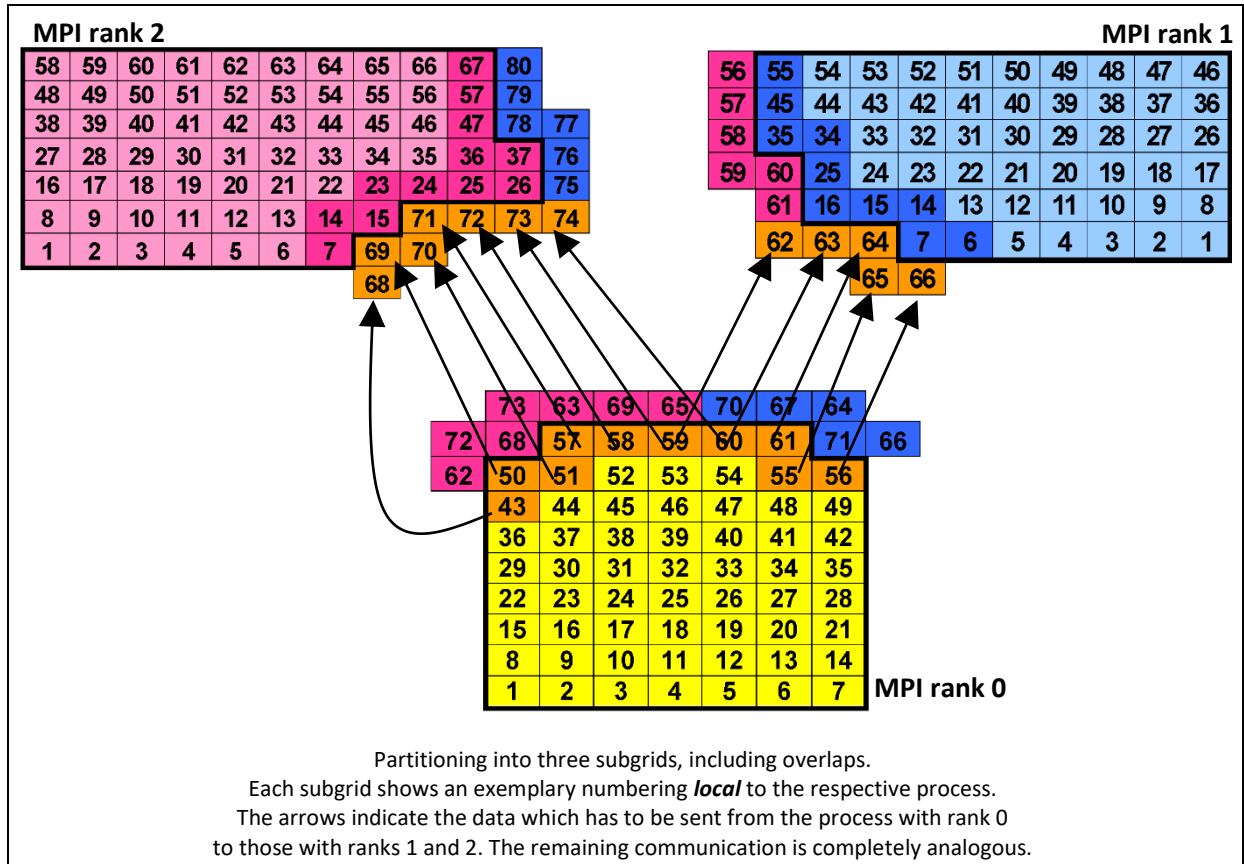
Parameter	Explanation
Local problem size	
nsys	Integer. Number of unknowns in the given system over all processes , ≥ 1 .
nnu	Integer. Number of (local) variables, ≥ 1 .
nna	Integer. Number of matrix entries stored in the (local) vector a , $\geq nnu$.
Input parameters involved in sending data	
npsnd	Integer. Total number of neighboring processors to which the current processor has to send data. Each of these neighbors obtains a subset of the <i>inner</i> halo area of the current processor.
iranksnd	Integer vector <i>iranksnd</i> ($1:npsnd$). Internally to the current processor, the neighboring processors to which data has to be sent are numbered $1, \dots, npsnd$. <i>iranksnd</i> has to provide the mapping between these numbers and the true MPI ranks. That is, for the i -th neighboring processor, $1 \leq i \leq npsnd$, <i>iranksnd</i> (i) has to equal its <i>MPI rank</i> .
nshalo	Integer. Total number of variables which have to be sent by the current processor to its neighbors during one exchange step. Only needed for dimensioning the below vector <i>isndlist</i> . Remark: <i>nshalo</i> amounts to the number of variables of the <i>inner</i> halo area of the current processor, counted according to their multiplicity in being sent .
The following two integer vectors point to the variables which really have to be sent. The use of <i>ipts</i> and <i>isndlist</i> is completely analogous to that of <i>ia</i> and <i>ja</i> .	
ipts	Integer vector <i>ipts</i> ($1:npsnd+1$). Together with the next vector, <i>isndlist</i> , this defines the range of variables which has to be sent to each of the neighboring processors. Remark: It always holds <i>ipts</i> (1)= 1 and <i>ipts</i> ($npsnd+1$)- $1=nshalo$.
isndlist	Integer vector <i>isndlist</i> ($1:nshalo$) Mapping into the inner halo area of the current processor, that is, <i>isndlist</i> has values in $1, \dots, nnu$. For any $1 \leq i \leq npsnd$, <i>u</i> (<i>isndlist</i> (j)) with <i>ipts</i> (i) $\leq j \leq$ <i>ipts</i> ($i+1$)- 1 specifies the range of variables of the inner halo area which is to be sent to the i -th neighboring processor. Remark: Since there may be variables which are to be sent to more than one processor, <i>isndlist</i> (j) may contain the same variable for different j's .
Input parameters involved in receiving data	
nprec	Integer. Total number of neighboring processors from which the current processor receives data for updating its <i>outer</i> halo area. Each of these neighbors sends values corresponding to subsets of the <i>outer</i> halo area of the current processor.
irankrec	Integer vector <i>irankrec</i> ($1:nprec$). Internally to the current processor, the neighboring processors from which data has to be received are numbered $1, \dots, nprec$. <i>irankrec</i> has to provide the mapping between these numbers and the true MPI ranks. That is, for the i -th neighboring processor, $1 \leq i \leq nprec$, <i>irankrec</i> (i) has to equal its <i>MPI rank</i> .
nrhalo	Integer. Total number of variables contained in the <i>outer</i> halo area of the current processor. Only needed for dimensioning the below vector <i>ireclist</i> .

The following two integer vectors point to the variables which really have to be received. The use of <i>iptr</i> and <i>ireclist</i> is completely analogous to that of <i>ia</i> and <i>ja</i> .	
iptr	Integer vector <i>iptr</i> (1:nprec+1). Together with the next vector, <i>ireclist</i> , this defines the range of variables which has to be received from each of the neighboring processors. Remark: It always holds <i>iptr</i> (1)=1 and <i>iptr</i> (nprec+1)-1 =nrhalo.
ireclist	Integer vector <i>ireclist</i> (1:nrhalo). Mapping into the outer halo area of the current processor (in local numbering), that is, <i>ireclist</i> has values in nnu+1,..., nnu+nrhalo. For any $1 \leq i \leq nprec$, <i>u</i> (<i>ireclist</i> (<i>j</i>)) with <i>iptr</i> (<i>i</i>) ≤ <i>j</i> ≤ <i>iptr</i> (<i>i</i> +1)-1 specifies the range of variables of the outer halo area which is being received from the <i>i</i> -th neighboring processor. Remark: All variables in the receive list <i>ireclist</i> (<i>j</i>) are required to occur exactly once.
Local part of the linear system to be solved	
ia, ja, a	Integer vectors <i>ia</i> (1:nnu+1), <i>ja</i> (1:nna); double vector <i>a</i> (1:nna). CSR representation of local part of matrix A. Note: nna=ia(nnu+1)-1.
u	Double vector <i>u</i> (1:nnu+nrhalo), local solution vector. <i>The auxiliary space (nnu+1 to nnu+nrhalo) does not need to be initialized.</i>
f	Double vector <i>f</i> (1:nnu+nrhalo), local part of the right hand side. <i>The auxiliary space (nnu+1 to nnu+nrhalo) does not need to be initialized.</i>
Local variable-to-unknown and variable-to-point pointers	
iu, ndiu	Integer vector <i>iu</i> (1:ndiu), variable-to-unknown pointer. Only relevant if nsys>1 in which case ndiu=nnu+nrhalo. Otherwise, <i>iu</i> is just a dummy vector and ndiu=1 .
ip, ndip	Integer vector <i>ip</i> (1:ndip), variable-to-point pointer. Only relevant if nsys>1 and if any of the point-based approaches is to be used. Then ndip=nnu+nrhalo. Otherwise, <i>ip</i> is just a dummy vector and ndip=1 .

3.2.1 Example of Input Data in the Local Numbering Case with SAMGp

In the below example we assume that three processors (rank 0, 1 and 2) are involved in the parallel computation on a T-shaped grid with the three processor areas being disjointly separated by fat lines. Each cell stands for a variable and the matrix connectivity structure is assumed to correspond to that of a compact *9-point discretization*, that is, each variable (cell) is connected to all of its eight neighbors except near (physical) boundaries where certain connections are simply missing.





The following table displays all MPI-relevant parameters for this example, assuming that the **current processor** is the one with **MPI rank 0**.

We first observe that $nnu=61$ and $nna=486$. Furthermore, we point out that in this exemplary situation $nshalo/ipts$ and $nrhalo/iptr$ are identical. This cannot be expected in general, though.

Parameter	Value	Comment
nshalo	12	The <i>inner</i> halo area of the rank 0 processor contains 10 variables (colored in orange). Except for 59 & 60, all of them are to be sent to just one proc (either rank 1 or rank 2). Variables 59 & 60, however, need to be sent to both processors, i.e., they have a multiplicity of 2. Consequently, $nshalo=12$.
npsnd	2	Number of neighboring processors to which the current processor has to send data.
iranksnd	2,1	MPI ranks of the neighboring processors.
ipts	1,8,13	Range of indices of variables to be sent.
isndlist	43,50,51,57,58, 59,60, 59,60,61,55,56	Indices of variables which are to be sent to the two neighboring procs: pink and blue indices correspond to the variables to be sent to the processors with rank 2 and 1, respectively.
nrhalo	12	Number of variables in the <i>outer</i> halo area. This area consists of all <i>numbered</i> variables in the dark pink and dark blue area.
nprec	2	Number of neighboring processors from which the current processor has to receive data.
irankrec	2,1	MPI ranks of the neighboring processors.
iptr	1,8,13	Range of indices of variables to be received.

ireclist	62,72,68,73,63, 69,65, 70,67,64,71,66	Indices of variables which are to be received from the two neighboring procs: pink and blue indices correspond to the variables to be received from the processors with rank 2 and 1, respectively.
-----------------	---	---

Attention: The user has to ensure that the list of data sent by processor i to processor j is **consistent** with the list of data received by processor j from processor i ! For instance, in the above example the **send list** from the process with *rank 0* to that with *rank 2* is: 43,50,51,57,58,59,60. Reversely, the corresponding **receive list** of the process with rank 2 is: 68,69,70,71,72,73,74.



4 File Interface: Main Files

If you request technical support (for instance, to help optimizing SAMG's performance for your type of application), we suggest that you send us some concrete linear system of equations, $Au=f$, which is typical for your type of application. While we in principal can also read common matrix file formats such as the matrix market format, these formats do typically not cover all relevant information that has been discussed before. This is especially true for the coupled system case. Thus, based on the data structure described in the previous section, there is a file format that can cover all relevant data.

We make the convention that all files which correspond to the same linear system of equations have the same **root name** and differ only in their suffixes. In the following, to be specific, let us assume the root name to be "**mydata**".

4.1 Format of Writing: Formatted or Binary

SAMG supports formatted and binary files. The data format is exactly the same and binary writing just means that all files – except for the *frm* and *siz* file – are written byte-by-byte. Which type of writing was used will be specified in the *frm* file, as we will describe in the following Section 4.2.1.

Within this manual we will consider formatted writing in order to illustrate the data format. For data exchange, however, the binary format has clear advantages:

- it uses the minimum possible amount of disk space,
- it is (most likely) the fastest one,
- in contrast to formatted I/O, it is not prone to produce round-off,

Ways to achieve binary writing: There are Fortran compilers which directly allow for binary writing⁴, in which case you just need to open the respective files with "form=binary". Alternatively, you may use binary C++ I/O.

Important: By default, we assume *Little Endian*, 4-byte integer and 8-byte floating point number representations. Should you use any other representation in providing your data to us, please precisely specify all differences.



4.2 Scalar Systems

In case of scalar systems two files, *mydata.frm* and *mydata.amg*, are mandatory. Optionally, these may be complemented by the files *mydata.rhs*, *mydata.sol* and *mydata.1st*.

4.2.1 Mandatory Files

mydata.frm: This file contains global information and has to be written as follows (*io* = I/O unit). Either (case of **formatted** writing of the matrix data):

```
write(io,'(a)') 'f    4'5
write(io,*) nna,nnu,matrix,1,0
```

or (case of **binary** writing of the matrix data):

```
write(io,'(a)') 'b    4'5
write(io,*) nna,nnu,matrix,1,0
```

⁴ For example, Intel compilers. The only thing which then has to be agreed upon between "Writer" and "Reader" is the "Endianness". Intel compilers, e.g., support both Endiannesses. Fehler! Textmarke nicht definiert.

⁵ In the case of using 64bit integers, please use the matrix format identification 14 instead of 4.

where nnu and nna denote the total numbers of variables and matrix entries, respectively. The integer value *matrix* gives some information on the type of matrix:

<i>matrix</i> =11:	A is symmetric and rowsum zero ⁶
<i>matrix</i> =12:	A is symmetric and <i>not</i> rowsum zero
<i>matrix</i> =21:	A is non-symmetric and rowsum zero
<i>matrix</i> =22:	A is non-symmetric and <i>not</i> rowsum zero

mydata.amg: This file contains the matrix *A* in compressed row format (→ Section 2.1), written as follows:⁷

```
do i=1,nnu+1; write(io,*) ia(i); enddo
do i=1,nna;   write(io,*) ja(i); enddo
do i=1,nna;   write(io,*) a(i); enddo
```

4.2.2 Optional Files

mydata.rhs: This file contains the *right hand side* vector *f*, written as follows:⁷

```
do i=1,nnu; write(io,*) f(i); enddo
```

mydata.sol: This file contains the *solution* vector (computed up to machine accuracy), written in the same format as **mydata.rhs**.

mydata.1st: This file contains the *1st approximation*, written in the same format as **mydata.rhs**.

4.2.3 Example

For the following linear system

$$\begin{pmatrix} 2 & -1 & & \\ -1 & 2 & -1 & \\ & -1 & 2 & -1 \\ 2 & & -1 & 2 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} 1.0 \\ 2.0 \\ 3.0 \\ 4.0 \end{pmatrix},$$

the above mentioned files look as follows

mydata.frm:	<i>f</i>	4			
	11	4	22	1	0

mydata.amg:	1
	3
	6
	9
	12
	1
	2
	2
	1
	3
	3
	2
	4
	4
	3
	1

⁶ **Rowsum zero** means that *all* rows sum up to zero. Otherwise, at least one row sum has to differ from zero.

⁷ Note that this type of writing implies that there will be just *one entry per row*.

	+2.0
	-1.0
	+2.0
	-1.0
	-1.0
	+2.0
	-1.0
	-1.0
	+2.0
	-1.0
	+2.0

mydata.rhs:	1.0
	2.0
	3.0
	4.0

4.3 Coupled Systems

In case of a coupled system ($nsys > 1$), the description of the previous section carries over, except that the file `mydata.frm` has to be slightly extended. Moreover, the variable-to-unknown and (optionally) variable-to-point vectors have to be provided as two files `mydata.iu` and `mydata.ip`, respectively.

mydata.frm: For **coupled systems**, this file now has to be written as follows:

```
write(io,'(a)') 'f    4'
write(io,*) nna,nnu,matrix,nsys,npnt
do n=1,nsys; write(io,*) iscale(n); enddo8
```

The parameters `nna`, `nnu` and `matrix` are as before. `nsys` denotes the number of unknowns. The integer switch `npnt` has to be 1 or 0, depending on whether or not the optional file `mydata.ip` exists. The vector `iscale(1:nsys)` has values 0 or 1, indicating which unknowns require scaling (here, an unknown is said to require scaling if it is determined only up to a constant): the k -th unknown requires scaling if `iscale(k)=1`, otherwise not.

mydata.amg, **mydata.rhs**, **mydata.sol**, **mydata.1st**: As before.

mydata.iu: Contains the variable-to-unknown pointer vector `iu` (see Section 2.2.1), written as⁸

```
do i=1,nnu; write(io,*) iu(i); enddo
```

mydata.ip: Contains the variable-to-point pointer vector `ip` (see Section 2.2.2), written as⁸

```
do i=1,nnu; write(io,*) ip(i); enddo
```

4.4 Distributed Data in MPI Parallel Runs

If a user is working in a distributed MPI environment, he might wish to send a matrix problem in distributed form. We support a distributed file format which is a natural extension of the serial format explained before.

Our file format requires the matrix to be available in a “globally split” form. That is, every MPI process comprises certain complete rows of the matrix, and each row sits on exactly one process. Analogously,

⁸ Note that this type of writing implies that there will be just *one entry per row*.

this is assumed to be true also for all involved vectors such as `f`, `u`, and eventually `iu`, `ip`, `coo`⁹). Of course, the same index distribution is assumed for all data objects. Importantly, the `ja(.)` indices are assumed to be global, that is, a process' portion is a set of rows identically occurring in the serial global matrix¹⁰. Consequently, every process can write its data to files in exactly the same format as explained before. For instance, a process would write its `.amg` file as follows:

```
do i=1,nnu+1; write(io,*) ia(i); enddo
do i=1,nna;   write(io,*) ja(i); enddo
do i=1,nna;   write(io,*) a(i); enddo
```

with `nnu`, `nna` being the sizes local to the respective process. Note that, as already stated further above, the `ja(.)` indices correspond to a global numbering and certain values `ja(i)` refer to variables which are actually located in different processes.

For each of the standard AMG files of type `.amg`, `.rhs`, etc. there now have to be as many files as there are processes, `p`, say. The names of these files are the same, except that the process numbers, `0,...,p-1` have to be encoded consistently in the process' file names. For example, the `.amg` files might be

`mydata_000.amg`, `mydata_001.amg`,

where the position of the numbering field is arbitrary, as is the field width¹¹.

In addition to the files per process, a single formatted file `mydata.siz` needs to be provided, containing `p+1` lines, line 1 containing just the number `p`, line `1+i` containing `nnu` and `nna` of process `i-1` (`i=1,...,p`). For instance, with a serial matrix of size `nnu=98`, `nna=1444`, in a split case with `p=4`, the `.siz` file could look like

```
4
24 320
26 420
24 384
24 320
```

4.5 Coordinates

Where geometric coordinates are available, they might be supplied in addition to the mere matrix data, as they can help analyzing a problem. If, however, geometry-based optimizations of SAMG are to be exploited, these information are mandatory, of course.

mydata.coo: This file has to be written as follows:¹²

```
write(io,*) nnu,ndim
do i=1,nnu
  write(io,*) x,y,z ! skip "z" if ndim=2
enddo
```

where `nnu` is as before and `ndim` is the spatial dimension of the application (i.e., 2 or 3).

⁹ The `coo` file will be described in Section 4.5 below.

¹⁰ The row indices possessed by a process don't need to be consecutive.

¹¹ Thus, the file naming could as well be something like `data_00_msys.amg`, `data_01_msys.amg`, or also with left adjusted numbers like in `dat_0_xx.amg`, `dat_1_xx.amg`, ..., `dat_10_xx.amg`,

¹² Note that there will be just one pair/triple of coordinates per row.

5 Index

.

.1st 14
 .amg 14
 .coo 16
 .frm 13, 15
 .ip 15
 .iu 15
 .rhs 14
 .siz 16
 .sol 14

6

64bit integer 13

A

a 5, 7, 8, 10

C

complex-valued problems 6
 compressed row format 5
 coupled system 4, 6

D

data structure 5
 diagonal entries, sign of 5

E

endianess 13

F

f 5, 7, 8, 10

G

global numbering 8

I

ia 5, 7, 8, 10
 ip 6, 7, 8, 10
 iptr 10
 ipts 9

irankrec 9
 iranksnd 9
 ireclist 10
 isndlist 9
 iu 6, 7, 8, 10

J

ja 5, 7, 8, 10

L

local numbering 9

M

matrix, parameter 14, 15
 MPI 8

N

ndip 7, 8, 10
 ndiu 7, 8, 10
 nna 5, 7, 8, 9
 nnu 5, 7, 8, 9
 npnt 15
 nprec 9
 npsnd 9
 nrhalo 8, 9
 nshalo 9
 nsys 6, 7, 8, 9
 numbering, global 8
 numbering, local 9

O

ordering, pointwise 6

P

point 4

R

root name 13
 rowsum zero 14

S

scalar system 4

U

u 5, 7, 8, 10
unknown 4

variable-to-point pointer 8, 10
variable-to-point pointer, ip 6, 7
variable-to-unknown pointer 8, 10
variable-to-unknown pointer, iu 6, 7

V

variable 4

Z

zero rowsum 14